

Programa · Comisión 2026

AI Engineering

Clase 2:

Encadenamiento lógico: orquestación con LangChain

Objetivos

1. Componer procesos de IA con la sintaxis pipe (|) de LCEL para pipelines legibles y mantenibles.
2. Implementar el protocolo Runnable para operaciones síncronas, asíncronas y por lotes.
3. Gestionar ChatPromptTemplates dinámicos para desacoplar la lógica del negocio del modelo.
4. Definir contratos de datos estrictos con Pydantic y with_structured_output().
5. Configurar estrategias de resiliencia: retries con backoff exponencial y fallbacks.

Agenda

1. Arquitectura de cadenas con LCEL y Runnables.
2. Validación estructurada y resiliencia en cadenas.
3. Repaso: orquestación y contratos de datos.
4. Pre-Entrega 2: tu pipeline de procesamiento validado.

**¡Esta clase va a ser
grabada!**



01

LCEL y Runnables

¿Por Qué LCEL?

De código espagueti a pipelines

Con SDKs básicos, cada paso requiere manejo manual: formatear variables, llamar al modelo, parsear la respuesta.

Sumar streaming, ejecución en paralelo o tracing vuelve ese código exponencialmente complejo: código espagueti.

LCEL transforma esos procesos aislados en pipelines declarativas, robustas y listas para producción.

El Protocolo Runnable

La interfaz universal

- **invoke():** llama al componente con una entrada única, de forma síncrona.
- **ainvoke():** la versión asíncrona de invoke, para no bloquear el event loop.
- **batch():** llama al componente con una lista de entradas, optimizando la latencia total.

Analogía: cada Runnable es una estación de una fábrica; el operador | es la cinta que las conecta.

El Operador Pipe |

prompt | model | parser

PromptTemplate: toma un diccionario de variables y genera el mensaje formateado (Input → PromptValue).

ChatModel: toma ese mensaje y lo envía al LLM, devolviendo un objeto Message.

OutputParser: toma la respuesta cruda del modelo y la limpia (Message → String o JSON).

¿Qué Pasa Bajo el Capó?

`chain.ainvoke()` paso a paso

El diccionario de entrada fluye al prompt, que genera un objeto `ChatPromptValue`.

Ese objeto pasa al model, que lo entiende y genera un `BaseMessage`.

El `BaseMessage` pasa al parser, que extrae el contenido (`.content`) y devuelve un string limpio.

`chain.ainvoke({"language": "Spanish", "text": "..."})` ejecuta las tres estaciones en un solo paso.

Prompts como Software

Roles en el Prompt

ChatPromptTemplate define roles System, Human y AI, clave para instrucciones de seguridad y formato.

Input Shaping

RunnablePassthrough o funciones lambda inyectan y transforman datos antes de llegar al prompt.

Funciones como Runnables

LCEL convierte cualquier función en un Runnable automáticamente, sin código extra.

Tipos de Runnables

RunnableLambda

Convierte cualquier función Python en un Runnable, sin código adicional.

RunnablePassthrough

Deja pasar el input sin modificar, o solo agrega nuevas claves al diccionario.

RunnableSequence

Es lo que se arma automáticamente al encadenar pasos con el operador |.

Streaming en LCEL

Tokens que llegan, no que se esperan

Streaming envía tokens a medida que se generan, en vez de esperar la respuesta completa del modelo.

TTFT (Time To First Token): el tiempo desde que se envía la solicitud hasta que llega el primer token.

Si el modelo soporta streaming, LCEL lo expone automáticamente con `.stream()` o `.astream()`, sin configurar nada extra.

Serie o Paralelo

Secuencial

Cada paso espera al anterior. Menor complejidad, pero mayor latencia y costo total.

Paralelo

Los pasos independientes corren a la vez. Mayor throughput y escalabilidad.

El resultado

Paralelo cuando las tareas son independientes; secuencial cuando una depende de la otra.



Momentos de la Clase:

Actividad colaborativa

Para pensar

Para recordar

Coder Tips

Pensá en un flujo de tu día a día con varios pasos dependientes (pedir turno, confirmar, recordar). ¿Cuáles podrían correr en paralelo y cuáles deben ser secuenciales?

Práctica - Unidad 1

Refactorización a LCEL Asíncrono

Objetivo: migrar tu cliente imperativo del Módulo 1 a una cadena LCEL asíncrona.

Manos a la obra:

- Reemplazá la llamada manual al SDK por un ChatOpenAI o ChatAnthropic de LangChain.
- Componé prompt | modelo | StrOutputParser() y ejecutá con await chain.ainvoke({...}).

Resultado: un script que imprime texto plano, armado enteramente con el operador |.

02

Validación y Resiliencia

El Contrato de Datos

De texto libre a esquemas

Un LLM en texto libre puede cambiar el formato, alucinar claves JSON o devolver tipos incorrectos.

Pydantic define un esquema formal que el modelo está obligado a seguir: sintáctico, estructural y semántico.

`with_structured_output()` usa Tool Calling o JSON Mode del proveedor para que el modelo encaje en tu molde.

Texto vs. JSON

Texto Libre

Riesgo de que el modelo alucine claves, cambie el formato o devuelva un tipo de dato incorrecto.

Salida Estructurada

Pydantic fuerza un esquema formal; `with_structured_output()` lo garantiza a nivel de API.

El resultado

Un contrato de datos predecible, listo para conectar con el resto de tu aplicación.

Tipos de Error

No todos los fallos son iguales

Errores Transitorios: caídas de red, latencia o límites de tasa (429 Rate Limit) del proveedor.

Errores de Formato: el modelo no logró seguir exactamente el esquema Pydantic solicitado.

Un sistema de IA en producción no solo debe ser correcto: debe ser robusto ante ambos tipos de falla.

Estrategias de Recuperación

Retries y Fallbacks

Frente a un error, el sistema tiene dos estrategias complementarias de recuperación.

- **Retries con Backoff Exponencial:** si falla, **esperará cada vez un poco más antes de reintentar.**
- **Esto evita saturar el servicio y permite que fallos momentáneos se resuelvan solos.**
- **Fallbacks:** si un modelo falla repetidamente, **redirigí la carga a uno más ligero o a una respuesta predefinida.**

Combinar ambas estrategias es lo que separa un prototipo de un sistema listo para producción.

Errores de Arquitectura

No todos los fallos se reintentan

Confiar ciegamente en el JSON: nunca asumas que un `json.loads` funcionará. Usá siempre un validador de esquema.

Reintentar errores permanentes: un error lógico del prompt no se arregla reintentando 10 veces.

- Diferenciá errores de transporte (red, rate limit) de errores de lógica: cada uno pide una solución distinta.

Un buen logging te dice en segundos a cuál de los dos te enfrentás.

Tipos de Prompt Templates

Prompt Estático

Texto fijo sin variables. Simple y rápido, pero no reutilizable ni dinámico.

ChatPromptTemplate

Estructura conversacional con roles y placeholders. Ideal para la mayoría de los casos.

Few-Shot Prompt

Incluye ejemplos dentro del prompt. Mejora resultados en tareas complejas, a mayor costo.

Anatomía de un Prompt

Un prompt profesional no es un string

Rol (System): quién es el modelo y con qué expertise responde.

Contexto y Objetivo: qué información tiene disponible y qué se espera que logre.

Restricciones y Formato Esperado: qué no debe hacer, y en qué estructura debe responder.

Output Parsers: Comparativa

StrOutputParser

Extrae el texto plano desde un AI Message. Simple y ligero, sin validación de estructura.

JsonOutputParser

Convierte la respuesta en un dict JSON. Falla si el modelo no devuelve un JSON válido.

PydanticOutputParser

Valida contra un modelo Pydantic completo: tipado fuerte y validación automática.

Prompt Engineering: Técnicas

Zero-Shot y Few-Shot

Resolver sin ejemplos, o con unos pocos ejemplos que guían el formato de la respuesta.

Chain-of-Thought

Pedirle al modelo que razone paso a paso mejora resultados en problemas complejos.

Role Prompting

Asignarle un rol específico al modelo mejora la claridad y consistencia de sus respuestas.

Errores en Parsers

Cuando el modelo no sigue el formato

Validación: Pydantic detecta si la estructura o los tipos de datos no son correctos.

Retry o Reparse: se reintenta la llamada, o se le pide al modelo que corrija su propia respuesta.

Fallback: si todo falla, se usa un parser alternativo o una respuesta por defecto, sin romper el flujo.

Cache en LCEL

Cache de Prompts

Guarda respuestas exactas para inputs idénticos, evitando llamar al modelo de nuevo.

Cache Semántico

Usa embeddings para encontrar respuestas a preguntas similares, no solo idénticas.

El límite

Menor latencia y costo, a cambio de consistencia, vencimiento y tamaño del cache a gestionar.

Momentos de la Clase:

Para recordar

Coder Tips

Material ampliado

Cuestionario de tarea

#FindTheBug

La palabra clave del día es
Resiliencia: no alcanza con que la
cadena funcione una vez, tiene
que sobrevivir a un rate limit real
en producción.

Práctica - Unidad 2

Validación estructurada y resiliencia

Objetivo: convertir una cadena frágil en un pipeline resiliente con salida validada.

Manos a la obra:

- Definí EntityExtraction en Pydantic y aplicá llm.with_structured_output(EntityExtraction).
- Envolvé el modelo con .with_retry(stop_after_attempt=3, wait_exponential_jitter=True).

Resultado: una cadena prompt | resilient_llm que nunca devuelve un objeto sin validar.

Break,

andá por un cafecito
o mate, ¡y volvemos!



03

Repaso: Orquestación y Contratos

Runnable: la Ventaja Clave

No es solo sintaxis

La ventaja técnica de Runnable no es la sintaxis: es la interfaz común que estandariza invoke, batch, stream y ainvoke.

Gracias a esa interfaz, streaming y ejecución asíncrona se propagan automáticamente por toda la cadena.

LCEL es declarativo, no imperativo: busca reducir el código de pegamento, no reemplazar el análisis del arquitecto.

El Orden Importa

Prompt

Prepara y formatea la instrucción con las variables de entrada.

Model

Procesa esa instrucción y genera una respuesta en texto libre.

Parser

Transforma el texto libre en algo útil. Sin él, tu app recibe un AIMessage crudo.

Forma vs. Lógica

Lo que el esquema no te da gratis

Validación de Forma: JSON parseable, campos presentes y tipos correctos. Los Structured Outputs modernos ya lo resuelven.

Validación Semántica: que los datos tengan sentido de negocio, como que una fecha de inicio sea anterior a la de fin.

La forma se resuelve con el esquema; el sentido de negocio requiere un `model_validator` en Pydantic.

Los Structured Outputs son un piso, no el techo de tu validación.



Cuando Falla la Validación

Dejá que el modelo se corrija

Tratar un error de validación como crítico y frenar el sistema es un error frecuente.

La mejor práctica: capturar el error y reenviarlo al modelo dentro de un nuevo prompt, para que se autocorrija.

- Esto aumenta drásticamente la tasa de éxito, a diferencia de ignorar el error o reintentar sin cambiar nada.
- El propio error de Pydantic ya te dice qué campo falló: es información valiosa para el próximo intento.

Este patrón, error-aware retry, es distinto de un retry ciego que repite exactamente la misma llamada.

Observabilidad

LangSmith y la trazabilidad

Traces: el registro completo de una ejecución, de punta a punta.

Runs: cada ejecución individual dentro de un pipeline, con su propio input y output.

Spans: operaciones o nodos específicos dentro de un run, medibles por separado.

Métricas Clave de una Chain

TTFT y Latencia

Tiempo al primer token, y tiempo total de respuesta de principio a fin.

Cost y Throughput

Costo estimado por ejecución, y cantidad de solicitudes procesadas por segundo.

Success Rate y Retries

Tasa de éxito del pipeline, y cuántos reintentos hicieron falta para lograrlo.

Testing de Chains

Una cadena también se testea

Unit Testing: probar componentes aislados, como un parser o un prompt, por separado.

Integration Testing: probar la cadena completa de punta a punta, con datos reales.

Golden Datasets: conjuntos de datos de referencia para detectar regresiones cuando algo cambia.

Checklist

1. Logging estructurado en cada paso de la cadena.
2. Retry con backoff, y un Fallback definido si todo falla.
3. Timeout configurado en cada llamada al modelo.
4. Streaming activado donde mejora la experiencia del usuario.
5. Cache donde tenga sentido, para reducir costo y latencia.
6. Observabilidad con LangSmith, y métricas de TTFT, costo y throughput.
7. Versionado de prompts y tests automáticos antes de cada deploy.

Anti-patrones de LCEL

Prompt Gigante

Un solo prompt con múltiples objetivos es difícil de testear, versionar y mantener.

Sin Output Parser

Dejar que tu app reciba un AIMessage crudo en vez de datos limpios y tipados.

Todo Secuencial

Encadenar tareas independientes una por una, en vez de paralelizarlas con `RunnableParallel`.

PE

Pre-Entrega 2

¿Qué Debes Construir?

Pre-Entrega 2

Debés construir un Pipeline de Extracción de Entidades Técnicas a partir de texto sin procesar.

- **Esquema Pydantic:** `tecnologias` (lista), `nivel_de_criticidad` (enum `baja/media/alta`), `resumen_tecnico` (string).
- **Cadena LCEL:** Prompt + LLM + Output Parser, usando `with_structured_output()`.

Resiliencia: al menos un reintento automático si el LLM devuelve un JSON mal formado o incompleto.

Tu Misión

El entregable

 **Entrega: repositorio de GitHub con el modelo Pydantic, el prompt template y la cadena LCEL con reintentos**

 **Debe incluir un mini-script de prueba asíncrono**

Construí un pipeline que reciba texto libre y devuelva siempre un objeto validado, no importa cuán ambiguo sea el input.

Pasos

1. Definí tu contrato: creá la clase Pydantic con las restricciones que necesites (ej. lista no vacía).
2. Preparé el parser con `.with_structured_output()` sobre tu modelo de LangChain.
3. Ensamblá la cadena: `chain = prompt | model.with_structured_output(TuEsquema)`.
4. Añadí resiliencia envolviendo la llamada con `.with_retry()`.
5. Prueba de estrés: pasá un texto ambiguo y verificá si el validador lanza excepciones o el modelo se recupera.

Errores a Evitar

Antes de entregar, revisá esto

Ignorar el `finish_reason`: el LLM puede cortar la respuesta por falta de tokens sin que lo notes.

- Hardcoding de prompts: evitá F-strings sueltas; usá ChatPromptTemplate para mantener la modularidad.
- Olvidar el `await` al usar `ainvoke`: te vas a quedar con una coroutine en vez del resultado real.
- Que las llaves `{variable}` del prompt no coincidan con las claves del diccionario que le pasás.

¿Cómo se evalúa?

· 100 pts

Esquema y Salida Estructurada · 30%

Uso de Pydantic para definir contratos de datos e implementación de `with_structured_output`.

Cadena LCEL y Prompting · 30%

Uso de la sintaxis pipe (|) y diseño de `ChatPromptTemplate` para la extracción técnica.

Ejecución y Estructura · 40%

Uso de `.ainvoke()`, manejo de logs, y organización del repositorio con script de prueba.

Síntesis del módulo

De scripts a arquitectura

Pasamos de código imperativo con SDKs crudos a pipelines declarativas compuestas con el operador |.

- **El protocolo Runnable estandariza invoke, batch y stream; Pydantic convierte texto libre en contratos de datos.**
- **Retries y fallbacks transforman una cadena frágil en un sistema resiliente, listo para producción.**

Próximo módulo: le damos memoria de largo plazo a estas cadenas con embeddings y bases de datos vectoriales.

Orquestación Maestra

Sesión en Vivo 2

Analizaremos 'Cadenas Rotas': identificaremos por qué fallan arquitecturas LCEL mal diseñadas.

- **Design Sprint grupal:** diseñarán una cadena lógica para un caso de uso empresarial complejo.
- **Live Coding:** refactorización en vivo de una cadena básica a una profesional con reintentos y validación.

Traé tu código de la Pre-Entrega 2, lo vamos a usar como base para un ejercicio de Stress Test.

¿Preguntas?

De prompts sueltos a arquitectura de datos para IA. Soltá tus dudas del módulo.



Glosario

1. LCEL: LangChain Expression Language, lenguaje declarativo para componer cadenas con el operador `|`.
2. Runnable: interfaz común que implementan modelos, prompts y parsers en LangChain.
3. `invoke()` / `ainvoke()`: llaman a un Runnable de forma síncrona o asíncrona con una entrada única.
4. `batch()` / `abatch()`: ejecutan un Runnable sobre una lista de entradas, optimizando la latencia total.
5. `stream()`: devuelve fragmentos de la respuesta a medida que se generan.
6. `RunnableParallel`: ejecuta múltiples Runnables al mismo tiempo y combina sus resultados en un diccionario.

1. `RunnablePassthrough`: deja pasar o transforma datos dentro de la cadena antes de llegar al siguiente paso.
2. `ChatPromptTemplate`: define un prompt con roles (System, Human, AI) y variables dinámicas.
3. `StrOutputParser`: extrae el texto plano de la respuesta del modelo, en vez de un objeto `AIMessage`.
4. `Pydantic`: librería de Python para definir y validar esquemas de datos estrictos.
5. `with_structured_output()`: fuerza a un modelo a devolver una salida que cumple un esquema `Pydantic`.
6. `PydanticOutputParser`: parser alternativo que valida la salida del modelo contra un esquema `Pydantic`.

1. `with_retry()`: reintenta una llamada fallida, generalmente con backoff exponencial.
2. `with_fallbacks()`: define una cadena de respaldo si la principal falla repetidamente.
3. Backoff Exponencial: estrategia de reintento que espera cada vez más tiempo entre cada intento.
4. Tool Calling / JSON Mode: mecanismos del proveedor que garantizan una salida en formato estructurado.
5. `finish_reason`: campo que indica por qué el modelo cortó su respuesta (por ejemplo, falta de tokens).
6. LangSmith: herramienta de observabilidad para trazar cada paso de una cadena LCEL.

1. TTFT (Time To First Token): tiempo desde la solicitud hasta el primer token de la respuesta.
2. RunnableLambda: convierte cualquier función Python en un Runnable, sin código extra.
3. RunnableBranch: enruta condicionalmente según la lógica del input, dentro de una cadena.
4. RunnableSequence: la secuencia de Runnables que se arma automáticamente con el operador |.
5. RunnableAssign: agrega o actualiza claves de un diccionario existente dentro del flujo.
6. RunnableMap: mapea cada elemento de una lista a través de un Runnable.

1. Zero-Shot / Few-Shot: resolver una tarea sin ejemplos, o con unos pocos ejemplos guía.
2. Chain-of-Thought: técnica que pide al modelo razonar paso a paso antes de responder.
3. Role Prompting: asignarle un rol específico al modelo para mejorar la consistencia.
4. JsonOutputParser / StructuredOutputParser: parsers que fuerzan una salida en JSON o un esquema.
5. Trace / Span: el registro completo de una ejecución, y cada operación medible dentro de ella.
6. Golden Dataset: conjunto de datos de referencia usado para detectar regresiones en una cadena.

1. Cache Semántico: cache que usa embeddings para reconocer preguntas similares, no solo idénticas.
2. Throughput: cantidad de solicitudes o tokens procesados por unidad de tiempo.
3. TPS (Tokens por segundo): velocidad a la que un modelo genera tokens de salida.
4. Success Rate: proporción de ejecuciones de una cadena que terminan sin errores.
5. Unit / Integration Testing: probar componentes aislados, o la cadena completa de punta a punta.
6. Golden Path: la versión corregida y ejemplar de una cadena, usada como referencia de buenas prácticas.

¡Gracias, arquitectos de IA!

We have provided this presentation to you for informational and illustrative purposes on a confidential basis. We reserve the right to use of non-disclosure agreements (NDAs) to protect our privacy and intellectual property rights. The information contained in this document is highly sensitive, confidential and/or proprietary and is intended for the express use of the intended recipient, as denoted on the title page of this document. The recipient of this presentation agrees by its receipt not to reproduce, duplicate, or reveal, in whole or in part, information presented herein without written permission of Coderhouse. No representation or warranty, expressed or implied, is made as to the accuracy or completeness of the information contained in this presentation. Any quote contained herein is for estimation purposes. Estimates are based on current, known requirements. Actual estimates may change once project elements are finalized or negotiated. This document may contain confidential pricing information. All pricing is subject to change.

Copyright © 2026 Coderhouse. All rights reserved. Coderhouse and the Coderhouse logo are trademarks or registered trademarks of Coderhouse, in the U.S. and/or other countries. All other trademarks are the property of their respective owners.